

Explicación completa del código del reloj inteligente con Arduino

Documentación del proyecto

¡Qué bueno que ya funciona todo! Para presentarlo al profesor, lo más importante es que puedan explicar **la idea general del programa** y después qué hace cada bloque. El código está organizado como un sistema con varios módulos que trabajan juntos: reloj RTC, alarmas, pantalla, teclado, sensores, buzzer y memoria EEPROM.

1. Idea general del proyecto

El Arduino funciona como un reloj despertador inteligente:

- El **RTC DS3231** mantiene la hora aunque Arduino se apague.
- La **EEPROM del módulo ZS-042** guarda hasta cinco alarmas.
- El **keypad** permite navegar por los menús e ingresar números.
- El **LCD** muestra la hora, las alarmas, la temperatura y la humedad.
- El **DHT11** mide temperatura y humedad ambiental.
- El **KY-018** detecta luz y apaga el backlight cuando supera el umbral.
- El **buzzer** suena cuando alguna alarma coincide con la hora actual.

La lógica principal es que Arduino repite continuamente el `loop()`, revisando todos esos componentes.

2. Las librerías

Al comienzo aparecen:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
#include <DHT.h>
#include "RTClib.h"
```

Cada librería permite controlar un componente:

`Wire.h`

Permite la comunicación **I2C**, utilizada por el RTC, el LCD y la EEPROM.

`LiquidCrystal_I2C.h`

Controla el LCD mediante los pines SDA y SCL.

`Keypad.h`

Facilita la lectura del teclado matricial 4×4.

DHT.h

Permite leer el sensor DHT11.

RTClib.h

Permite comunicarse con el RTC DS3231 y trabajar con fechas y horas.

3. ¿Por qué varios dispositivos pueden compartir A4 y A5?

El LCD, el RTC y la EEPROM están conectados a:

SDA → A4

SCL → A5

Esto es posible porque utilizan el protocolo **I2C**.

Cada dispositivo tiene una dirección diferente:

LCD: 0x27

RTC: 0x68

EEPROM: 0x57

Arduino envía la información por los mismos cables, pero indica a qué dirección quiere comunicarse.

Es parecido a mandar cartas por una misma calle, pero cada casa tiene un número diferente.

4. Configuración del RTC

Esta línea crea el objeto que representa al reloj:

```
RTC_DS3231 rtc;
```

En el `setup()` se comprueba si Arduino puede comunicarse con él:

```
if (!rtc.begin()) {  
  lcd.print("No detecta RTC");  
  while (true) {  
    apagarBuzzer();  
  }  
}
```

`rtc.begin()` devuelve:

- `true` si encontró el RTC.
- `false` si no pudo comunicarse con él.

El signo `!` significa “no”. Por eso:

```
if (!rtc.begin())
```

se interpreta como:

Si el RTC no pudo iniciarse, mostrar un error.

El `while (true)` detiene el programa permanentemente porque sin RTC el reloj no podría funcionar correctamente.

5. Cómo se obtiene la hora real

La función principal es:

```
void actualizarHoraDesdeRTC() {
    DateTime ahora = rtc.now();

    horas = ahora.hour();
    minutos = ahora.minute();
    segundos = ahora.second();

    if (segundos != ultimoSegundoLeido) {
        ultimoSegundoLeido = segundos;
        revisarAlarmas();
    }
}
```

Esta línea:

```
DateTime ahora = rtc.now();
```

le pregunta al RTC cuál es la fecha y hora actual.

Después se separan sus partes:

```
horas = ahora.hour();
minutos = ahora.minute();
segundos = ahora.second();
```

Antes, el reloj se incrementaba con `millis()`. Ahora ya no hace falta contar manualmente los segundos, porque el DS3231 lleva el tiempo por sí mismo.

¿Para qué sirve `ultimoSegundoLeido`?

```
int ultimoSegundoLeido = -1;
```

Después:

```
if (segundos != ultimoSegundoLeido) {
    ultimoSegundoLeido = segundos;
    revisarAlarmas();
}
```

El `loop()` se ejecuta miles de veces por segundo. Sin esta condición, `revisarAlarmas()` también se ejecutaría miles de veces durante el mismo segundo.

Con `ultimoSegundoLeido`, las alarmas se revisan solamente una vez cada vez que cambia el segundo.

Ejemplo:

```
segundos actuales = 25  
último segundo = 24
```

Como son diferentes, se revisan las alarmas y se guarda:

```
último segundo = 25
```

Durante el resto del segundo 25, ya no vuelve a revisarlas.

6. Cómo se guarda una nueva hora en el RTC

La función es:

```
void guardarHoraEnRTC(int h, int m, int s) {  
    DateTime ahora = rtc.now();  
  
    rtc.adjust(  
        DateTime(  
            ahora.year(),  
            ahora.month(),  
            ahora.day(),  
            h,  
            m,  
            s  
        )  
    );  
}
```

Primero lee la fecha actual:

```
DateTime ahora = rtc.now();
```

Después utiliza:

```
rtc.adjust(...)
```

para cambiar la hora del RTC.

Se conserva:

```
ahora.year()  
ahora.month()  
ahora.day()
```

y solamente se reemplazan:

```
h  
m  
s
```

Así no se pierde la fecha que ya estaba guardada.

7. Diferencia entre RTC y EEPROM

Esta diferencia es muy importante para la presentación.

RTC DS3231

Guarda y mantiene:

- Hora
- Minutos
- Segundos
- Fecha

Tiene una pila que permite que el reloj siga avanzando aunque Arduino esté apagado.

EEPROM AT24C32

Guarda datos creados por ustedes:

- Si una alarma está activa.
- Hora de la alarma.
- Minutos de la alarma.

La EEPROM no cuenta el tiempo. Solamente conserva información.

Por eso:

- El **RTC** conserva la hora actual.
- La **EEPROM** conserva las alarmas.

8. Cómo están representadas las cinco alarmas

Se utilizan tres arreglos:

```
int alarmaHoras[MAX_ALARMAS];
int alarmaMinutos[MAX_ALARMAS];
bool alarmaActiva[MAX_ALARMAS];
```

Como:

```
const int MAX_ALARMAS = 5;
```

cada arreglo tiene cinco posiciones.

Ejemplo:

Índice	0	1	2	3	4
Alarma visible	1	2	3	4	5
Hora	07	09	14	18	22
Minuto	30	00	15	45	10
Activa	sí	no	sí	sí	no

La alarma 1 está en la posición 0, porque los arreglos de C++ empiezan desde cero.

Por eso aparece:

```
int indice = numero - 1;
```

Si el usuario elige la alarma 3:

```
numero = 3  
indice = 2
```

9. Cómo se guarda cada alarma en la EEPROM

Cada alarma utiliza tres bytes:

Byte 1: activa o desactivada

Byte 2: hora

Byte 3: minuto

La dirección inicial es:

```
#define EEPROM_ALARMAS_ADDR 10
```

Y para calcular la posición de cada alarma:

```
unsigned int base = EEPROM_ALARMAS_ADDR + (indice * 3);
```

Por ejemplo:

Alarma 1 → posiciones 10, 11 y 12

Alarma 2 → posiciones 13, 14 y 15

Alarma 3 → posiciones 16, 17 y 18

Alarma 4 → posiciones 19, 20 y 21

Alarma 5 → posiciones 22, 23 y 24

Después se escriben los valores:

```
epromWriteByte(base, alarmaActiva[indice] ? 1 : 0);  
epromWriteByte(base + 1, alarmaHoras[indice]);  
epromWriteByte(base + 2, alarmaMinutos[indice]);
```

Esta expresión:

```
alarmaActiva[indice] ? 1 : 0
```

es un operador ternario.

Significa:

```
if (alarmaActiva[indice]) {  
    guardar 1;  
} else {  
    guardar 0;  
}
```

10. ¿Qué es el valor “magic”?

El código utiliza:

```
#define EEPROM_MAGIC_ADDR 0
#define EEPROM_MAGIC_VALUE 123
```

Cuando la EEPROM se usa por primera vez, sus valores pueden ser desconocidos.

El programa guarda el número 123 en una posición especial:

```
EEPROM_WriteByte(EEPROM_MAGIC_ADDR, EEPROM_MAGIC_VALUE);
```

Luego, al encender:

```
byte magic = EEPROM_ReadByte(EEPROM_MAGIC_ADDR);
```

```
if (magic != EEPROM_MAGIC_VALUE) {
    inicializarAlarmasEnRAM();
}
```

Esto permite saber si la EEPROM ya fue inicializada por el programa.

Es como una firma que dice:

Esta memoria ya contiene datos válidos de nuestro sistema.

11. Cómo se comprueba si una alarma debe sonar

La función recorre las cinco alarmas:

```
void revisarAlarmas() {
    for (int i = 0; i < MAX_ALARMAS; i++) {
```

La condición principal es:

```
if (
    alarmaActiva[i] &&
    horas == alarmaHoras[i] &&
    minutos == alarmaMinutos[i] &&
    segundos == 0 &&
    !alarmaYaDisparoEsteMinuto[i]
) {
    alarmaSonando = true;
    alarmaYaDisparoEsteMinuto[i] = true;
}
```

Para que suene, deben cumplirse todas estas condiciones:

1. La alarma está activa.
2. La hora actual coincide.
3. Los minutos coinciden.
4. Los segundos son cero.
5. Esa alarma todavía no sonó durante ese minuto.

El operador:

&&

significa “y”.

Todas las condiciones deben ser verdaderas.

12. ¿Por qué existe alarmaYaDisparoEsteMinuto?

Sin esta variable, la alarma podría activarse varias veces en el mismo minuto.

```
bool alarmaYaDisparoEsteMinuto[MAX_ALARMAS];
```

Cuando una alarma se activa:

```
alarmaYaDisparoEsteMinuto[i] = true;
```

Después, cuando ya no coincide la hora o el minuto:

```
if (
    horas != alarmaHoras[i] ||
    minutos != alarmaMinutos[i]
) {
    alarmaYaDisparoEsteMinuto[i] = false;
}
```

El operador:

```
||
```

significa “o”.

Cuando el reloj sale del horario de la alarma, se vuelve a habilitar para el próximo día.

13. Funcionamiento del buzzer invertido

El buzzer está conectado de una forma particular:

5V → buzzer → pin 11

Por eso funciona al revés:

- LOW: suena.
- HIGH: se apaga.

La variable:

```
const bool BUZZER_INVERTIDO = true;
```

le indica al programa qué tipo de conexión se está utilizando.

La función para encenderlo es:

```
void encenderBuzzer() {
    if (BUZZER_INVERTIDO) {
        digitalWrite(BUZZER_PIN, LOW);
    } else {
        digitalWrite(BUZZER_PIN, HIGH);
    }
}
```

```
}  
}
```

Así, el mismo código podría funcionar con las dos formas de conexión.

14. Cómo se produce el sonido intermitente

La función es:

```
void gestionarSonidoAlarma() {  
    if (alarmaSonando) {  
        if (millis() - previoMillisBuzzer >= 300) {  
            previoMillisBuzzer = millis();  
            estadoBuzzer = !estadoBuzzer;  
        }  
    }  
}
```

Cada 300 milisegundos se invierte:

```
estadoBuzzer = !estadoBuzzer;
```

Si era true, pasa a false; si era false, pasa a true.

Después:

```
if (estadoBuzzer) {  
    encenderBuzzer();  
} else {  
    apagarBuzzer();  
}
```

Esto produce:

```
300 ms encendido  
300 ms apagado  
300 ms encendido  
300 ms apagado
```

La ventaja de usar millis() es que no se bloquea todo el programa con un delay(300).

15. Máquina de estados del menú

El programa utiliza:

```
enum Estados {  
    MOSTRAR_RELOJ,  
    MENU_PRINCIPAL,  
    MENU_ALARMAS,  
    SET_HORA,  
    SET_ALARMA,  
    VER_ALARMAS,  
    BORRAR_ALARMA,  
    MOSTRAR_CLIMA
```

```
};
```

Esto se llama **máquina de estados**.

La variable:

```
Estados estadoActual = MOSTRAR_RELOJ;
```

indica en qué pantalla está el usuario.

Después, en el `loop()`:

```
switch (estadoActual) {
```

se ejecuta únicamente el bloque correspondiente al estado actual.

Ejemplo:

```
case MOSTRAR_RELOJ:
```

muestra el reloj.

```
case MENU_ALARMAS:
```

muestra las opciones de alarmas.

```
case MOSTRAR_CLIMA:
```

muestra temperatura y humedad.

Esto evita tener todo mezclado en una sola función.

16. Cómo se reemplaza una alarma

Cuando se elige un número del 1 al 5, el programa revisa:

```
if (alarmaActiva[indice]) {
```

Si ya hay una alarma activa en esa posición, muestra su horario:

```
A2 08:30
```

```
#:Reemp *:Salir
```

Después espera una decisión:

```
char confirmacion = esperarTeclaValida('#', '*');
```

- #: reemplaza la alarma.
- *: cancela.

Cuando se carga la nueva hora, se sobrescriben los valores anteriores:

```
alarmaHoras[indice] = hh;
```

```
alarmaMinutos[indice] = mm;
```

```
alarmaActiva[indice] = true;
```

Y después se actualiza la EEPROM:

```
guardarAlarmaEnEEPROM(indice);
```

17. Cómo se borra una alarma

Para borrarla, no hace falta eliminar físicamente los bytes. Se marca como inactiva:

```
alarmaActiva[indice] = false;
alarmaHoras[indice] = 0;
alarmaMinutos[indice] = 0;
```

Después se escribe nuevamente en la EEPROM:

```
guardarAlarmaEnEEPROM(indice);
```

Al quedar `alarmaActiva` en `false`, la función `revisarAlarmas()` la ignora.

18. Lectura del DHT11

La función utiliza:

```
float nuevaHumedad = dht.readHumidity();
float nuevaTemperatura = dht.readTemperature();
```

Los valores son `float` porque pueden tener decimales.

Después se revisa que la lectura sea válida:

```
if (
    !isnan(nuevaHumedad) &&
    !isnan(nuevaTemperatura)
)
```

NaN significa **Not a Number**.

El DHT devuelve NaN cuando no logra realizar una lectura válida.

Por eso, el programa solamente actualiza los datos si ambos valores son correctos:

```
humedad = nuevaHumedad;
temperatura = nuevaTemperatura;
```

19. ¿Por qué el DHT se lee cada dos segundos?

```
const unsigned long intervaloDHT = 2000;
```

El DHT11 es un sensor lento. No conviene consultarlo constantemente.

Esta condición:

```
if (millis() - previoMillisDHT >= intervaloDHT)
```

hace que se lea aproximadamente cada dos segundos.

Además, permite que Arduino continúe atendiendo el reloj, las alarmas y el teclado durante ese tiempo.

20. Funcionamiento del sensor de luz

Se lee con:

```
valorLuz = analogRead(SENSOR_LUZ);
```

`analogRead()` devuelve un valor entre:

0 y 1023

Después se compara con el umbral:

```
if (valorLuz > UMBRAL_LUZ) {  
    lcd.noBacklight();  
} else {  
    lcd.backlight();  
}
```

En este proyecto:

```
const int UMBRAL_LUZ = 120;
```

Si supera 120, se apaga el backlight. Si vuelve a bajar, se enciende.

El sensor no apaga todo el LCD: solamente controla la luz de fondo. El texto sigue estando en la pantalla, aunque sea difícil verlo sin iluminación.

21. Qué ocurre dentro del `loop()`

La estructura general es:

```
void loop() {  
    actualizarHoraDesdeRTC();  
    leerSensorDHT();  
    controlarLuzLCD();  
  
    char tecla = teclado.getKey();  
  
    gestionarSonidoAlarma();  
  
    switch (estadoActual) {  
        ...  
    }  
}
```

En cada repetición Arduino:

1. Lee la hora del RTC.
2. Revisa si debe sonar una alarma.
3. Actualiza el DHT11 cuando corresponde.
4. Lee el sensor de luz.

5. Comprueba si se presionó una tecla.
6. Controla el buzzer.
7. Muestra la pantalla correspondiente.

Esta repetición ocurre continuamente mientras Arduino está encendido.

Explicación breve para presentar oralmente

Podrían decir algo como:

Nuestro proyecto utiliza un RTC DS3231 para mantener la hora aunque Arduino se apague. Las cinco alarmas se almacenan en la memoria EEPROM AT24C32 del módulo ZS-042, donde cada alarma ocupa tres bytes: estado, hora y minuto. El programa utiliza una máquina de estados para controlar las diferentes pantallas del menú. Una vez por segundo compara la hora del RTC con todas las alarmas activas. Cuando encuentra una coincidencia, activa el buzzer de manera intermitente hasta que el usuario presiona una tecla. También incorporamos un DHT11 para mostrar temperatura y humedad, y un sensor KY-018 que controla automáticamente la luz de fondo del LCD.